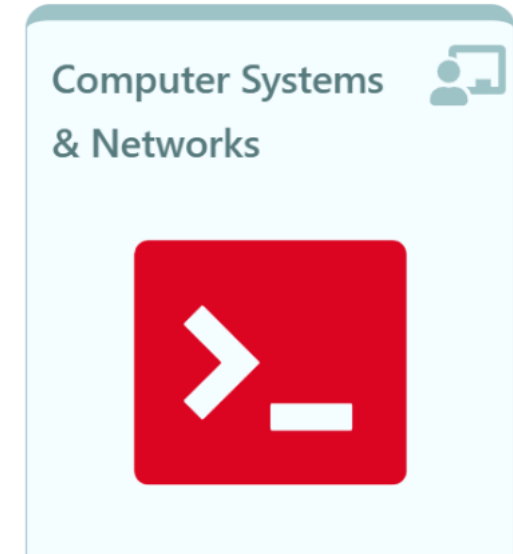


Computer Systems & Networks



01: Search and Filter



GREP

sort · cut · grep

02: Regular Expressions



basic RegEx · extended RegEx · metacharacters

03: Positional Parameters



\$0 \$1 \$2
cp file1.txt file2.txt
\$@

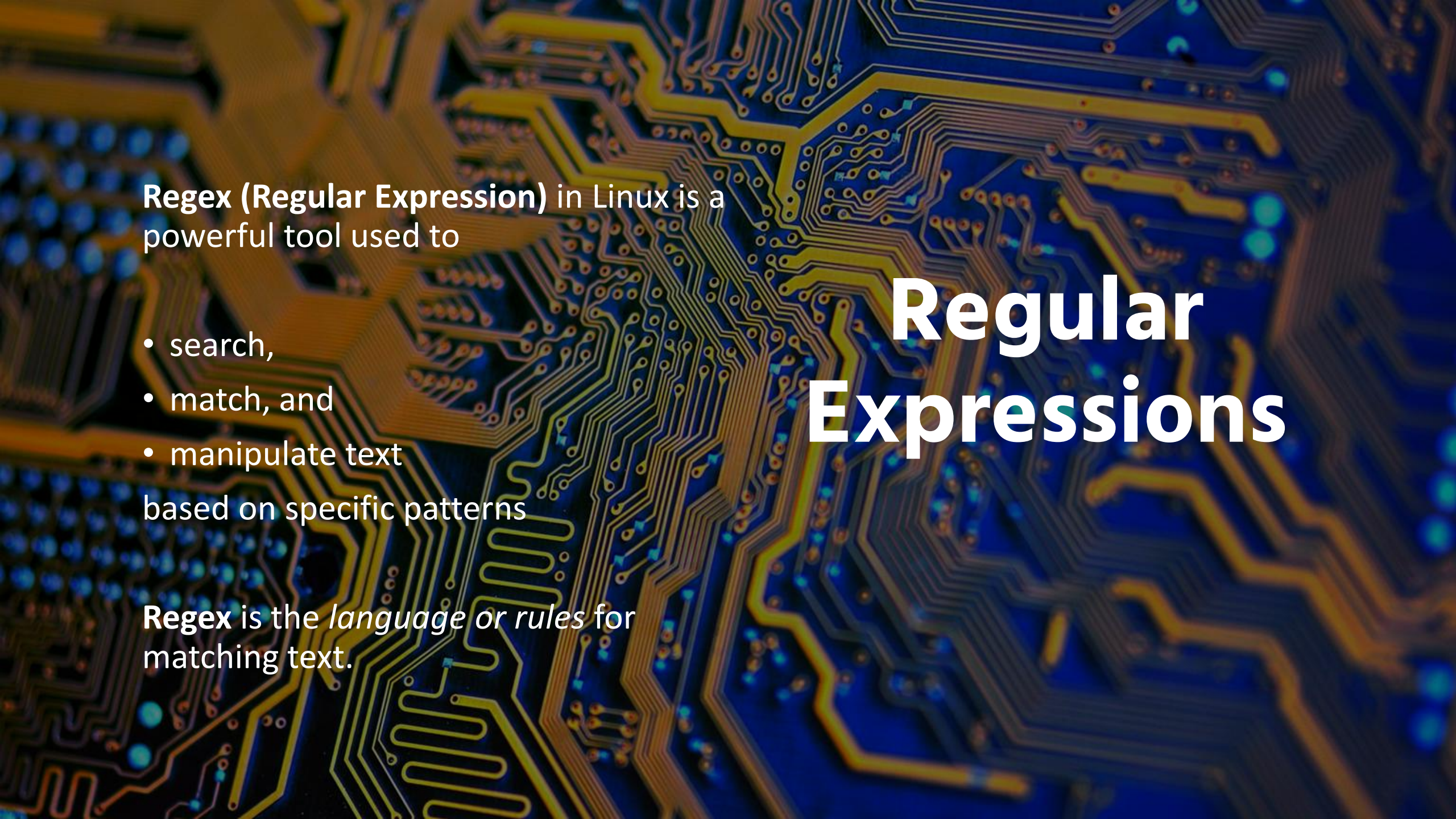
Positional Parameters · exit

Lab



```
#!/bin/bash
```

grep · RegEx · Variables ·
Positional Parameters · Exit



Regex (Regular Expression) in Linux is a powerful tool used to

- search,
 - match, and
 - manipulate text
- based on specific patterns

Regex is the *language or rules* for matching text.

Regular Expressions

Regex is used in many common Linux tools:

Command	Description
grep	Search text using patterns
sed	Stream editor (find & replace, etc.)
awk	Pattern scanning and processing
find	Find files based on name patterns

Done: previous notes

Examples

Regex is the *language or rules* for matching text.

Grep is the *program* that uses those rules to search files.

→ You give **grep** a **regex pattern**, and it searches for lines that match it.

```
$ grep "hello" file.txt      #basic search
```

grep → command you're using

hello → regex pattern (matches lines that matches "hello")

file.txt → file to search in

Examples

Regex is the *language or rules* for matching text.

Grep is the *program* that uses those rules to search files.

→ You give **grep** a **regex pattern**, and it searches for lines that match it.

\$ grep "hello" file.txt	#basic search
\$ grep "^start" file.txt	# ^ match Beginning of Line
\$ grep "end\$" file.txt	# \$ match End of Line
\$ grep "a.*z" file.txt	# Matches lines with an "a", followed by any/no characters (.*), and ending in "z" (somewhere later in the line)
\$ grep "a.*z\$" file.txt	# what's the output?

Regular Expressions(regex)

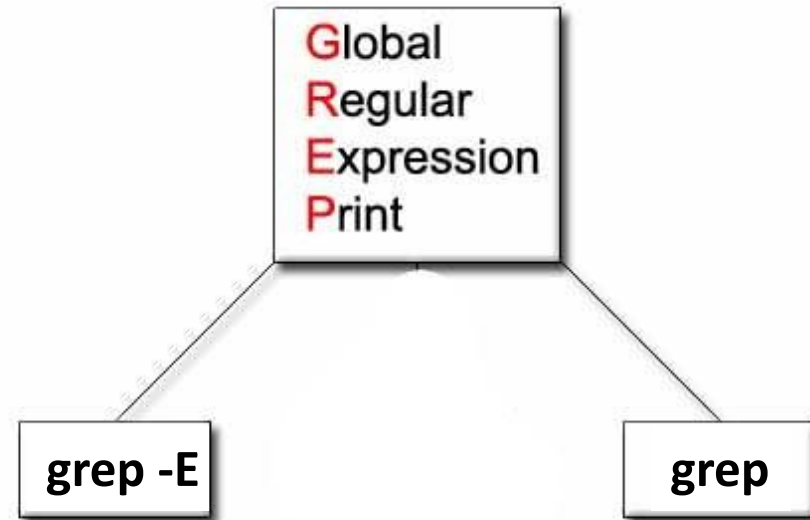
Regex are a collection of *normal* and *special* characters used to find simple or complex patterns, respectively, in files.

Normal characters are alphanumeric characters which match themselves. E.g. an **a** character would match an **a**

Metacharacters/Special characters have special meanings when used within patterns by commands

. * ? ^

```
grep 'example' fileName  
grep 'example' file1 file2 file3  
grep 'string1 string2' fileName  
cat otherfile | grep 'example2'  
command | grep 'example2'  
command option1 | grep 'data'  
grep --color 'data' filename
```



Basic RegEx - The . Character

```
$ grep 'r..f' red.txt  
reef  
roof
```

The . character matches any character except for the new line character.

The pattern **r..f** would find any line that contained the letter **r** followed by exactly two characters and then the letter **f**

→ The . character can be used any number of times.

Basic RegEx - The [] Character

[] match a single character from the list or range of possible characters contained within the brackets:

[0-9]

match all lines containing a **digit**

[aeiou]

match all lines containing a **vowel**.

[^xyz]

A carat can be used at the beginning of the range to specify a **negative** range.

[^xyz] will match all lines that do not contain x, y or z

Basic RegEx - The * Character

* is used to match **zero or more** occurrences of a character or pattern preceding it.

```
sysadmin@localhost:~/Documents$ grep 're*d'  
red.txt  
red  
reeed  
rd
```

For example, **e*** would match zero or more occurrences of the letter **e**:

It is also possible to match zero or more occurrences of *a list of* characters using the square brackets

```
sysadmin@localhost:~/Documents$ grep 'r[oe]*d' red.txt
```

Basic RegEx - Anchor Characters

^ \$

Anchor characters specify whether the match occurs at the beginning of the line or the end of the line.

BEGINNING: The caret **^** character is used to ensure that a pattern appears at the beginning of the line

```
sysadmin@localhost:~/Documents$ grep '^root' passwd  
root:x:0:0:root:/root:/bin/bash
```

END: The second anchor character, **\$**, can be used to ensure a pattern appears at the end of the line:

```
sysadmin@localhost:~/Documents$ grep 'r$' alpha-first.txt  
B is for Bear  
F is for Flower
```

SUMMARY

Character Matches

.	Any single character
[]	A list or range of characters to match one character. If the first character is the caret ^, it means any character <i>not</i> in the list
*	The previous character repeated zero or more times
^	If the first character in the pattern, the pattern must be at the beginning of the line to match, otherwise just a literal ^
\$	If the last character in the pattern, the pattern must be at the end of the line to match, otherwise just a literal \$

Basic RegEx - The \ Escape Character

Use the backslash character `\` to match a character that happens to be a special Regular Expression character:

In the example below, the pattern `re*` matches an `r` (followed by zero or more of the letter `e`):

```
sysadmin@localhost:~/Documents$ grep 're*' newhome.txt  
**Beware** of the ghost in the bedroom.
```

To search for the asterisk `*` character, place a `\` before the `*` character:

```
sysadmin@localhost:~/Documents$ grep 're\*' newhome.txt  
**Beware** of the ghost in the bedroom.
```

Basic RegEx - The \ Escape Character

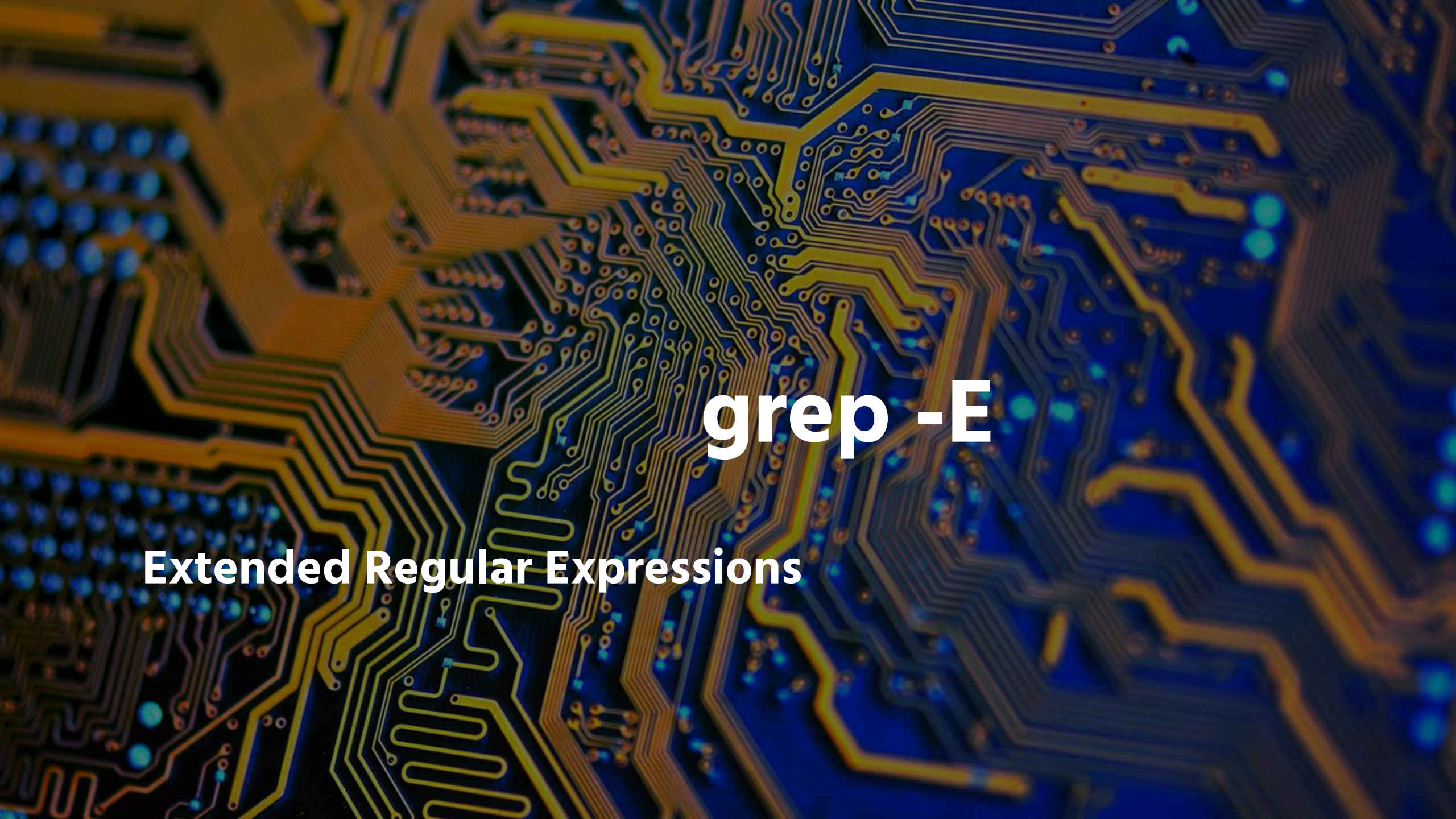
In grep, these are treated as pattern instead of meta characters:

? + { } | ()

By putting 'backslash' in front of meta character can let the grep to treat it as meta characters like

\? \+ \{ \| \(\ \)

Using **grep -E**, you wouldn't need to escape them



grep -E

Extended Regular Expressions

Regex Types in Linux



Basic Regular Expressions (BRE) – default in many tools like `grep` and `sed`

Special characters like `+`, `?`, `{}` are **not special** in BRE unless you **escape them** with a backslash (`\`)

EG: Match one or more would be written as `a\+`



Extended Regular Expressions (ERE) – used with **`grep -E`** and **`awk`**

Use Extended RegEx (`grep -E`, `sed -E`) when possible.

It's more powerful and readable because you don't have to escape so much

EG: Match one or more would be written as `a+`

Curly Brackets { }

Tells us about the number of occurrences of a character in a string

Expression	Description	EG
{n}	Matches the preceding character appearing 'n' times exactly	\$cat sample grep -E p{2}
{n,m}	Matches the preceding character appearing 'n' times but not more than m	\$grep -E "a{2,4}" file.txt
Matches lines that contain: "aa" (2 a's) "aaa" (3 a's) "aaaa" (4 a's) But not: "a" (1 a) "aaaaa" (5 a's or more)		

Curly Brackets { }

Tells us about the number of occurrences of a character in a string

Expression	Description	EG
{n}	Matches the preceding character appearing 'n' times exactly	\$cat sample grep -E p{2}
{n,m}	Matches the preceding character appearing 'n' times but not more than m	\$grep -E "a{2,4}" file.txt
{n, }	Matches the preceding character only when it appears 'n' times or more	\$grep -E "a{3,}" file.txt
	use grep -E to match lines where the letter a appears 3 or more times :	

Curly Brackets { }

Tells us about the number of occurrences of a character in a string

Expression	Description	EG
{n}	Matches the preceding character appearing 'n' times exactly	\$cat sample grep -E p{2}
{n,m}	Matches the preceding character appearing 'n' times but not more than m	\$grep -E "a{2,4}" file.txt
{n, }	Matches the preceding character only when it appears ' n' times or more	\$grep -E "a{3,}" file.txt
+	Matches one or more occurrence of the previous character	\$ cat employee.txt grep-E "w+a"

Output the contents of the file employee.txt, The pipe sends the output of cat to the next command (grep). Match **one or more** occurrences of the letter w immediately followed by the letter a

→ Lines that contain **at least one w followed immediately by an a**

Curly Brackets { }

Tells us about the number of occurrences of a character in a string

Expression	Description	EG
{n}	Matches the preceding character appearing 'n' times exactly	\$cat sample grep -E p{2}
{n,m}	Matches the preceding character appearing 'n' times but not more than m	\$grep -E "a{2,4}" file.txt
{n, }	Matches the preceding character only when it appears 'n' times or more	\$grep -E "a{3,}" file.txt
+	Matches one or more occurrence of the previous character	\$ cat employee.txt grep-E "w+a"
?	Matches zero or one occurrence of the previous character	\$grep -E "colou?r" file.txt

Extended RegEx

grep -E

Use

Use the ? character to match colo followed by zero or one u character followed by an r character:

```
sysadmin@localhost:~/Documents$ grep -E 'colou?r' spelling.txt
American English: Do you consider gray to be a color or a shade?
British English: Do you consider grey to be a colour or a shade?
```

Use

Use the + character to match one or more e characters:

```
sysadmin@localhost:~/Documents$ grep -E 'e+' red.txt
red
reef
reeed
```

Use

Use the | character to match either gray or grey:

```
sysadmin@localhost:~/Documents$ grep -E 'gray|grey' spelling.txt
American English: Do you consider gray to be a color or a shade?
British English: Do you consider grey to be a colour or a shade?
```

Summary

^	Indicates the beginning of an input string
\$	Indicates the end of the an input string
*	Indicates the preceding subexpression is to be matched zero or more times
+	Indicates the preceding subexpression is to be matched one or more times
?	Indicates the preceding subexpression is to be matched zero or one time
{n}	Match exactly n times (Where n is a non-negative integer)
{n,}	Match at least n times (Where n is a non-negative integer)
{n,m}	Match at least n and at most m times (Where m and n are non-negative integers and $n \leq m$)
.	Matches any single character except “n”
[xyz]	Match any one of the enclosed characters
x y	Match either x or y
[^xyz]	Match any character not enclosed
[a-z]	Matches any character in the specified range.
[^a-z]	Matches any character not in the specified range

Summary

b	Matches a word boundary (the position between a word and a space)
B	Matches a nonword boundary (example: 'uxB' matches the 'ux' in "tuxedo" but not the 'ux' in "Linux")
d	Matches a digit character
D	Matches a non-digit character
f	Matches a form-feed character
n	Matches a newline character
r	Matches a carriage return character
s	Matches any whitespace character (including space, tab, form-feed, etc.)
S	Matches any non-whitespace character
t	Matches a tab character
v	Matches a vertical tab character
w	Matches any word character including underscore
W	Matches any non-word character
un	Matches n, where n is a Unicode character expressed as four hexadecimal digits (such as, u0026 for the ampersand symbol)

Try out this weeks Lab

01: Search and Filter



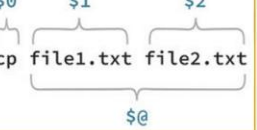
sort · cut · grep

02: Regular Expressions



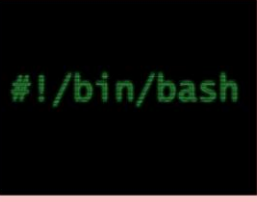
basic RegEx · extended RegEx · metacharacters

03: Positional Parameters



Positional Parameters · exit

Lab



grep · RegEx · Variables · Positional Parameters · Exit

 **Lab**
Computer Systems & Networks

Current Week
2

 Search

>  >  Computer Syste... >  02: File and T... >  02: Filtering ... >  Lab

Manipulate Files & Contents

01. sort, cut

02. grep

03. Variables

04. Positional Parameters

Cisco NetAcad

EXERCISES:

- Assume you wish to write a script called **numEmp**

```
compsys@compsys-virtualbox:~$ cat numE*  
echo "Today on `date` the total number of employees are"  
cat ~/employee.txt | wc -l
```

How could you improve on this to:

- Match all lines that start with **cat**
- Match all lines that end with **cat**
- Match all lines that contain any of the letters 'a', 'b' or 'c'
- Match all lines that do not contain a vowel
- Match all lines that start with a digit following zero or more spaces
- Match all lines that contain the word hello in upper-case or lower-case